

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

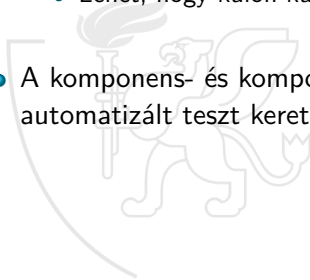
- A tesztelés a szoftverfejlesztés egy nagyon fontos fázisa, vagy inkább tevékenysége.
- Kérdés: Mi tartozik a teszteléshez, milyen tesztelési tevékenységek vannak?
 - A program követelményeinek, specifikációjának átnézése?
 - A program forráskódjának átnézése?
 - Program futtatása általános adatokkal?
 - Program futtatása speciális adatokkal?
 - Annak meghatározása, hogy a program mely részei futottak, és melyek nem?
 - Annak ellenőrzése, hogy a program elég gyors és könnyen használható-e?
 - A megtalált hibák összegyűjtése?
 - Hiba okának megkeresése?
 - Hiba kijavítása?
 - Hibajavítás után a program újraellenőrzése, hogy megjavult-e a hiba?
 - Hibajavítás után a program újraellenőrzése, hogy elromlott-e benne valami más?
 - Vélemény kialakítása a szoftver minőségéről?
 - Annak eldöntése, hogy elhalasztjuk a szoftver kiadását?
 - A tesztelés során végzett tevékenységek leírása a főnök számára?

- A tesztelés a szoftverfejlesztés egy nagyon fontos fázisa, vagy inkább tevékenysége.
- Mi tartozik a teszteléshez, milyen tesztelési tevékenységek vannak?
 - A program követelményeinek, specifikációjának átnézése
 - A program forráskódjának átnézése
 - Program futtatása általános adatokkal
 - Program futtatása speciális adatokkal
 - Annak meghatározása, hogy a program mely részei futottak, és melyek nem
 - Annak ellenőrzése, hogy a program elég gyors és könnyen használható-e
 - A megtalált hibák összegyűjtése
 - Hiba okának megkeresése
 - Hiba kijavítása
 - Hibajavítás után a program újraellenőrzése, hogy megjavult-e a hiba
 - Hibajavítás után a program újraellenőrzése, hogy elromlott-e benne valami más
 - Vélemény kialakítása a szoftver minőségéről
 - Annak eldöntése, hogy elhalasztjuk a szoftver kiadását
 - A tesztelés során végzett tevékenységek leírása a főnök számára

- A tesztelés [10, 11] a hibák felfedezésével
 - méri a szoftver aktuális minőségét,
 - hozzájárul a kockázatok csökkentéséhez, és
 - alapot ad a szoftver minőségének javításához.
- A tesztelés egy összetett folyamat, tevékenységek halmaza (menedzsment, a tesztelés megtervezése, tesztek megtervezése, implementációja, végrehajtása, ...), amelynek a szoftver ellenőrzése csak egy kis része.
- A tesztelés:
 - Felméri a szoftver minőségét, észleli a hibákat, meghibásodásokat, de önmagában nem javítja a szoftvert.
- Hibajavítás (debugging):
 - Ha már tudjuk, hogy rossz a szoftver, akkor megkeresi a hiba okát, és kijavítja a szoftvert.

- A tesztelés szintjei
 - Komponenstesztelés
 - Komponensintegrációs tesztelés
 - Rendszertesztelés
 - Rendszerintegrációs tesztelés
 - Elfogadási tesztelés
 - ...
- Tesztelési módszerek
 - Feketedoboz tesztelés
 - Fehérdoboz tesztelés
 - Tapasztalatalapú tesztelés
 - ...
- Tesztelési technikák
 - Ekvivalenciapartícionálás (EP)
 - Határérték-elemzés (BVA)
 - Döntési tábla tesztelés (DT)
 - ...
- Egyéb kategorizálás
 - Verifikáció
 - Validáció
 - Manuális tesztelés
 - Automatizált tesztelés
 - Ellenőrző tesztelés
 - Regressziós tesztelés
 - Statikus tesztelés
 - Dinamikus tesztelés
 - Karbantartási tesztelés
 - ...

- **Komponenstesztelés tesztelés**
 - A szoftver építőelemeinek önálló, a szoftver többi részétől elkülönített tesztelése.
 - Ha már a komponens sem működik jól, mit várunk attól a szoftvertől, amibe beépítjük?
 - Komponens alatt általában függvényt értünk (de OOP nyelvekben ez lehet pl. objektum is).
- **Komponensintegrációs tesztelés**
 - A szoftver építőelemeinek együttműködését teszteli.
 - Lehet, hogy külön-külön jól működnek a komponensek, de mégsem dolgoznak jól együtt.
- A komponens- és komponensintegrációs tesztek általában a fejlesztők készítik valamilyen automatizált teszt keretrendszer segítségével.



Tesztesetek

Mi a teszteset?

- Egy tesztesetnek öt fontosabb része van:
 - Előfeltétel: mikor, milyen környezetben indítható?
 - Bemenet: milyen tesztadatokat viszünk be a rendszerbe?
 - Lépések: mit kell tennünk a teszt során?
 - Elvárt kimenet: milyen adatokat várunk a végén?
 - Utófeltétel: mi változott a környezetben?
- Komponenstesztelnél, vagyis egy függvény tesztelésénél ezek nagyjából a következők:
 - Globális változók, erőforrások állapota, elérhetősége.
 - Argumentumok vagy egyéb módon átadott értékek.
 - Függvény meghívása.
 - Visszaadott vagy egyéb módon előállított (pl. kiírt) érték.
 - Globális változók, erőforrások állapota.
- Egy teszteset vagy átment (*pass*) vagy bukik (*fail*). Nincs olyan, hogy „részben sikerül”.

- A teszteset megvalósítása komponens- és komponensintegrációstesztnél általában a következőképpen néz ki:
 - Kód, amely ellenőrzi/beállítja a megfelelő környezetet.
 - Input értékek előállítása.
 - Függvény meghívása.
 - Kód, amely ellenőrzi és összehasonlítja a visszaadott értékeket az elvárt eredménnyel (`assert` segítségével).
 - Kód, amely a környezeti értékeket, az erőforrások állapotát ellenőrzi (`assert` segítségével).
- Ha szigorúan komponens (unit) teszteket készítünk, akkor a feladat valószínűleg legbonyolultabb része a kapcsolódó, nem tesztelendő unit-ok „levágása”.

- Hogyan válasszunk input értékeket a teszteléshez?
 - EP Ha valahogy csoportosítani tudjuk a lehetséges bemeneti vagy kimeneti értékeket, vagy a különböző működési módokat, akkor mindegyik csoporthoz válasszunk egy értéket.
 - BVA Ha az inputot intervallumokra tudjuk osztani a lehetséges működési módok szerint, akkor vegyük az intervallumok határértékeit, teszteljük ezekkel és a „külső” szomszédaikkal.
 - DT Ha a végeredményt egy több feltételből álló összetett logika állítja elő, akkor teszteljük a feltételek összes lehetséges kombinációját.

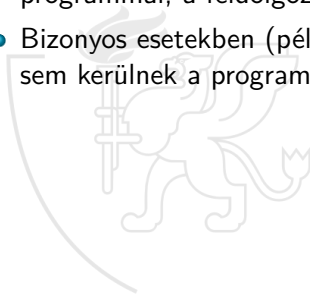


Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz tartozó műveletek
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban**
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Az assert utasítás

- `assert expression`
- `assert expression, message`
 - Ha az `expression` értéke hamis, akkor a program hibával leáll.
 - Ha van `message`, akkor a hibával együtt azt is látja a felhasználó.
- Az `assert` leginkább programírás közben, teszteléskor, debuggoláskor hasznos: a programmal, a feldolgozott adatokkal kapcsolatos feltételezéseket szokás vele ellenőrizni.
- Bizonyos esetekben (például a `-O` vagy `-OO` kapcsoló használatakor) az `assert`-ek bele sem kerülnek a program lefordított/futtatott verziójába.



Az assert utasítás

Példa

```
$ python3
```

```
>>> def func(n):  
...     assert n > 0, "Works only for positives"  
...     if n == 1:  
...         return 0  
...     return 1 + (func(3 * n + 1) if n % 2 == 1 else func(n // 2))  
...  
>>> func(-1)  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
  File "<stdin>", line 2, in function  
AssertionError: "Works only for positives"  
>>> func(21)  
7  
>>> func(31)  
106
```

Az assert utasítás

Példa

```
$ python3 -0
```

```
>>> def func(n):  
...     assert n > 0, "Works only for positives"  
...     if n == 1:  
...         return 0  
...     return 1 + (func(3 * n + 1) if n % 2 == 1 else func(n // 2))  
...  
>>> func(-1)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
File "<stdin>", line 5, in function
```

```
File "<stdin>", line 5, in function
```

```
File "<stdin>", line 5, in function
```

```
[Previous line repeated 995 more times]
```

```
File "<stdin>", line 3, in function
```

```
RecursionError: maximum recursion depth exceeded in comparison
```

Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - **A pytest**
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A pytest modul

- A `pytest` a Python egyik modulja, amely tulajdonképpen egy nagyon egyszerű egységteszt keretrendszert (*unit test framework*) valósít meg.
- Önálló programként `pytest`-ként futtatható.
 - A megadott könyvtárakban (vagy az aktuálisban, ha nem adunk meg egyet sem) megkeresi a `test_*.py` Python fájlokat, és tesztfájlként futtatja őket.
 - Megadhatunk neki közvetlenül bármilyen Python forrásfájlt is, ilyenkor ezeket is tesztfájlként kezeli.
 - A tesztfájlokban megkeresi a `test_` kezdetű függvényeket, majd egymás után tesztként lefuttatja őket.
 - Ezekben a tesztfüggvényekben `assert`-eket használhatunk ellenőrzésre, a bukó `assert` bukó tesztet jelent.
 - A keretrendszer összegyűjti a teszteredményeket, és riportot készít belőlük.
- Saját programból a `pytest` importja után a `pytest.main([a1, a2, ...])` hívásnak van olyan hatása, mintha a parancssorból a `pytest a1 a2 ...` utasítást adtuk volna ki.

Riadólánc tesztelése

test_riadolanc_ciklikus.py [1–25]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-12-01
6cc-by-nc-nd
7
8A ciklikus függvény tesztelése a riadólánc probléma megvalósításából.
9"""
10import pytest
11from riadolanc_ciklikus import ciklikus
12from sys import argv
13
14def test_ciklikus():
15    assert ciklikus([1, 2, 3, 0]) is True
16
17def test_tobb_ciklus():
18    assert ciklikus([1, 0, 3, 2]) is False
19
20def test_nem_teljes():
21    assert ciklikus([1, 2, 3, 1]) is False
22
23if __name__ == "__main__":
24    pytest.main(argv[:1])
```

Másodfokú egyenlet gyökeit megoldó függvény tesztelése

test_masodfok.py [10–34]

```
10 import pytest
11 from masodfok import megoldo
12 from sys import argv
13
14 def test_konstans_0():
15     assert megoldo(0, 0, 0) is True
16
17 def test_konstans_nem_0():
18     assert megoldo(0, 0, 1) is None
19
20 def test_linearis():
21     assert megoldo(0, 1, 2) == -2.0
22
23 def test_egy_gyok():
24     assert megoldo(1, -2, 1) == 1.0
25
26 def test_ket_gyok():
27     gyok = megoldo(1, -3, 2)
28     assert type(gyok) is tuple and set(gyok) == {1.0, 2.0}
29
30 def test_nem_valos():
31     assert megoldo(1, 0, 4) is None
32
33 if __name__ == "__main__":
34     pytest.main(argv[:1])
```

Másodfokú egyenlet gyökeit megoldó függvény tesztelése

test_masodfok_c.py [10–35]

```
10 import pytest
11 from masodfok_c import megoldo
12 from sys import argv
13
14 def test_konstans_0():
15     assert megoldo(0, 0, 0) is True
16
17 def test_konstans_nem_0():
18     assert megoldo(0, 0, 1) is None
19
20 def test_linearis():
21     assert megoldo(0, 1, 2) == -2.0
22
23 def test_egy_gyok():
24     assert megoldo(1, -2, 1) == 1.0
25
26 def test_ket_gyok():
27     gyok = megoldo(1, -3, 2)
28     assert type(gyok) is tuple and set(gyok) == {1.0, 2.0}
29
30 def test_nem_valos():
31     gyok = megoldo(1, 0, 4)
32     assert type(gyok) is tuple and set(gyok) == {2j, -2j}
33
34 if __name__ == "__main__":
35     pytest.main(argv[:1])
```